The Journal of Open Source Software

MooBench: A micro-benchmark for performance overhead of observability tools

David Georg Reichelt [1,2¶], **Shinhyung Yang** [3], **Marcel Hansson** [4], and **Wilhelm Hasselbring** [3]

1 Lancaster University Leipzig **2** Universitätsrechenzentrum (URZ) Leipzig **3** Kiel University **4** University of Hamburg ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

In partnership with

AMERICAN ASTRONOMICAL SOCIETY

This article and software are linked with research article DOI 10.3847/xxxxx <- update this with the DOI from AAS once you know it., published in the .

DRAFT

Summary

Understanding the runtime behavior of software is inherently difficult due to the unpredictability of the behavior itself and the non-determinism of underlying layers, such as Just-In-Time (JIT) compilation in virtual machines, operating system scheduling, and CPU frequency scaling. Observability tools aim to answer questions regarding runtime behavior of software, such as "How much time did this request take?" or "How often did method A call method B?" (Majors et al., 2022). These questions are answered using telemetry data, i.e., measurement data that is obtained from the code execution. To collect telemetry data, additional code needs to be executed, which introduces overhead. This overhead affects both system performance and the accuracy of the measurements themselves. The MooBench microbenchmark measures this overhead and contains factorial experiments that facilitate breaking down this overhead into its root causes.

The MooBench benchmark has been originally developed to examine the performance overhead of Kieker in Java and was extended as a general overhead measurement microbenchmark for various observability tools, currently within the Java and Python ecosystem. In this paper, we describe the MooBench microbenchmark.

Statement of need

Observability tools are crucial for managing software system health and optimizing operational costs; consequently, a wide variety of tools compete in the market (Siegfried et al., 2025). The relevance of distributed tracing tools increased with the rise of microservice architectures, where system behavior across microservices needs to be understood (Sigelman et al., 2010). While observabiliy generally consists of the three pillars traces, metrics, and logs, especially distributed tracing is gaining wide adoption (Janes et al., 2023).

In the context of microservices, various studies have examined the overhead of tracing. For example, Nou et al. (Nõu et al., 2025) investigated the overhead of OpenTelemetry and Elastic APM using open-source applications, and examined the root causes of the overhead using profiling. Ahmed et al. (Ahmed et al., 2016) compared the overhead of commercial observability tools and the open-source tool Pinpoint for regression detection. Eder et al. (Eder et al., 2023) compared the overhead using serverless applications.

Besides the microservice context, tracing overhead has been examined for operating systems and High-Performance Computing (HPC). For operating system, the overhead of tools for tracing the kernel itself (Desnoyers & Dagenais, 2006) (Gebai & Dagenais, 2018) and eBPF, that allows to trace both within the kernel and the user space (Volpert et al., 2025) (Domaschka et al., 2023) has been examined in case studies. In the context of HPC, research has addressed

the impact of overhead (Hunold et al., 2022) and the combination of instrumentation and sampling to reduce it (Ilsche et al., 2015).

The aforementioned studies utilize macrobenchmarks to examine overhead in realistic use cases. While macrobenchmarks provide an indication of overhead in similar scenarios, they often fail to isolate the baseline overhead or identify its specific sources. When looking at specific distributed cases, the overhead sources are influenced by the processes running in parallel, their scheduling, and by the network behavior. MooBench addresses this by providing a microbenchmark that measures the fundamental overhead of observability frameworks in a controlled environment. Using different configurations, it enables factorial experiments that isolate root causes of overhead, specifically distinguishing between instrumentation, data collection, and data serialization.

Software design

MooBench consists of two parts: the minimal and configurable system under test, and the automation of benchmarking the observability. These are described in the following.

System under test

The overhead of observability tools is caused by data collection. For tracing tools, this data collection is caused by method execution. The runtime behavior of the execution of methods is influenced by various factors, including class loading, Just-in-Time compilation (JIT), and garbage collection. To minimize these effects, MooBenchs SuT consists of only one method (`monitoredMethod`) that calls itself recursively `$RECURSION_DEPTH` times. Only this recursive calls could be removed by the JIT compiler, therefore, the last method call contains a busy wait for `$METHOD_TIME` nanoseconds. Figure (**?**) visualizes the call tree.

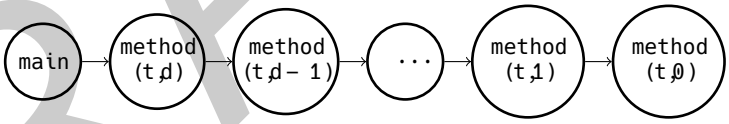

Figure 1: Call tree of the MooBench microbenchmark showing the recursive method calls to control the stack depth.

This system under test is currently implemented in Java and Python. It is planned to extend it for JavaScript and Go.

Observability automation

Each observability framework requires a different setup and has different capabilities. For example, OpenTelemetry can be attached to the JVM using `-javaagent`, requires the property otel.instrumentation.methods.include to be set to a list of classes, and can then export to Zipkin, Jaeger, or Prometheus. Furthermore, OpenTelemetry can be started with empty otel.instrumentation.methods.include or without export, which enables measuring the overhead of instrumentation without data collection or data collection without serialization. Due to these different configurations, it is necessary to implement configuration scripts for every framework individually. MooBench has these configuration scripts in `frameworks/$TECHNOLOGY-$LANGUAGE` (e.g., `frameworks/OpenTelemetry-java`). The scripts usually contain a `benchmark.sh` for starting the experiment, `functions.sh` with specific download or execution functions, `labels.sh` containing the names of the configurations and `config.rc` containing eventually necessary definitions of additional environment variables.

To measure performance in managed runtimes like the JVM, the managed runtime needs to be started multiple times, inside of the managed runtime, the workload needs to be repeated until

DRAFT

a given count of warmup iterations is finished, and finally, the measurement iterations within the managed runtime need to be executed (Georges et al., 2007). MooBench implements this process by executing `$NUM_OF_LOOPS` loops, where every loop runs all configurations of one framework. Inside of each run, `$TOTAL_NUM_OF_CALLS` defines the number of iterations, i.e., repetitions of all calls to `monitoredMethod`. The execution data are stored into csv files, and after the execution is finished, the warmup iterations are removed.

Research impact statement

There have been numerious case studies that examined overhead and its root causes.

Waller et al. (Waller et al., 2014)...

Eichelberger et al. (Eichelberger & Schmid, 2014)...

Knoche et al. (Knoche & Eichelberger, 2018)...

Reichelt et al. (Reichelt et al., 2021)...

Improvement Case Studies

Strubel et al. (Strubel & Wulf, 2016)...

Reichelt et al. (Reichelt et al., 2023)...

Reichelt et al. (Reichelt et al., 2024)...

Yang et al. (Yang et al., 2024)...

MooBench development papers

Should include (Yang et al., 2025), (Reichelt et al., 2024)?

AI usage disclosure

There is no known AI usage, however, AI usage in PRs is not monitored.

Acknowledgements

We acknowledge contributions from André van Hoorn, Reiner Jung, Thomas Düllmann, ...

References

Ahmed, T. M., Bezemer, C.-P., Chen, T.-H., Hassan, A. E., & Shang, W. (2016). Studying the effectiveness of application performance management (APM) tools for detecting performance regressions for web applications: An experience report. *Proceedings of the 13th International Workshop on Mining Software Repositories*, 1–12. https://doi.org/10.1145/2901739.2901774

Desnoyers, M., & Dagenais, M. R. (2006). The lttng tracer: A low impact performance and behavior monitor for gnu/linux. *OLS (Ottawa Linux Symposium)*, *2006*, 209–224.

Domaschka, J., Volpert, S., Maier, K., Eisenhart, G., & Seybold, D. (2023). Using eBPF for database workload tracing: An explorative study. *Companion of the 2023 ACM/SPEC International Conference on Performance Engineering*, 311–317.

DRAFT

Eder, C., Winzinger, S., & Lichtenthäler, R. (2023). A comparison of distributed tracing tools in serverless applications. *2023 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, 98–105. https://doi.org/10.1109/SOSE58276.2023.00018

Eichelberger, H., & Schmid, K. (2014). Flexible resource monitoring of Java programs. *Journal of Systems and Software*, *93*, 163–186.

Gebai, M., & Dagenais, M. R. (2018). Survey and analysis of kernel and userspace tracers on linux: Design, implementation, and overhead. *ACM Comput. Surv.*, *51*(2). https://doi.org/10.1145/3158644

Georges, A., Buytaert, D., & Eeckhout, L. (2007). Statistically rigorous java performance evaluation. *ACM SIGPLAN Notices*, *42*(10), 57–76. https://doi.org/10.1145/1297027.1297033

Hunold, S., Ajanohoun, J. I., Vardas, I., & Träff, J. L. (2022). An overhead analysis of mpi profiling and tracing tools. *Proceedings of the 2nd Workshop on Performance EngineeRing, Modelling, Analysis, and VisualizatiOn Strategy*, 5–13.

Ilsche, T., Schuchart, J., Schöne, R., & Hackenberg, D. (2015). Combining instrumentation and sampling for trace-based application performance analysis. *Tools for High Performance Computing 2014: Proceedings of the 8th International Workshop on Parallel Tools for High Performance Computing, October 2014, HLRS, Stuttgart, Germany*, 123–136.

Janes, A., Li, X., & Lenarduzzi, V. (2023). Open tracing tools: Overview and critical comparison. *Journal of Systems and Software*, *204*, 111793.

Knoche, H., & Eichelberger, H. (2018). Using the Raspberry Pi and Docker for Replicable Performance Experiments: Experience Paper. *Proc. 8th ACM/SPEC Int. Conf. On Performance Engineering*, 305–316.

Majors, C., Fong-Jones, L., & Miranda, G. (2022). *Observability engineering*. " O'Reilly Media, Inc.".

Nõu, A., Talluri, S., Iosup, A., & Bonetta, D. (2025). Investigating performance overhead of distributed tracing in microservices and serverless systems. *Companion of the 16th ACM/SPEC International Conference on Performance Engineering*, 162–166.

Reichelt, D. G., Jung, R., & Hoorn, A. van. (2024). Overhead measurement noise in different runtime environments. *arXiv Preprint arXiv:2411.05491*.

Reichelt, D. G., Kühne, S., & Hasselbring, W. (2023). Towards solving the challenge of minimal overhead monitoring. *Comp. 14th ACM/SPEC Int. Conf. On Performance Engineering*, 381–388.

Reichelt, D. G., Kühne, S., & Hasselbring, W. (2021). *Overhead comparison of OpenTelemetry, inspectIT and kieker*.

Siegfried, G., Crossley, M., Byrne, P., Bridges, A., & Caren, M. (2025). *Magic quadrant for observability platforms* [Research Report]. Gartner, Inc. https://www.gartner.com/doc/reprints?id=1-2LF3Y49A&ct=250709&st=sb

Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., & Shanbhag, C. (2010). *Dapper, a large-scale distributed systems tracing infrastructure*.

Strubel, H., & Wulf, C. (2016). Refactoring Kieker's monitoring component to further reduce the runtime overhead. *Symposium on Software Performance 2016 (SSP '16)*.

Volpert, S., Winkelhofer, S., Domaschka, J., & Wesner, S. (2025). Towards eBPF overhead quantification: An exemplary comparison of eBPF and SystemTap. *Companion of the 16th ACM/SPEC International Conference on Performance Engineering*, 122–129. https://doi.org/10.1145/3680256.3721311

DRAFT

Waller, J., Fittkau, F., & Hasselbring, W. (2014). Application performance monitoring: Trade-off between overhead reduction and maintainability. *Proceedings of the Symposium on Software Performance*.

Yang, S., Reichelt, D. G., & Hasselbring, W. (2024). Evaluating the overhead of the performance profiler cloudprofiler with MooBench. *arXiv Preprint arXiv:2411.17413*.

Yang, S., Reichelt, D. G., Ingo, H., & Hasselbring, W. (2025). Detection of performance changes in MooBench results using nyrki\" o on GitHub actions. *arXiv Preprint arXiv:2510.11310*.

DRAFT